

# MathSolver：基于 LLM 的微积分与概率论 智能解题与批改系统

项目报告

2026 年 6 月 28 日

## 摘要

MathSolver 是一个面向微积分与概率论领域的 LLM 智能体系统，遵循 ToRA 与 SelfCheck 范式，结合符号计算引擎 SymPy/scipy、MCP 外部工具调用，实现了数学问题的自动求解与逐步批改。系统采用 19 类问题模式（Schema）驱动的提示工程，支持单路径推理与 USC 多路径自治性采样，并在批改阶段引入 PARC 前提链验证与元验证机制以降低误判风险。本文从系统架构、核心模块和评测体系等方面对该项目进行说明。

**关键词：**LLM 智能体；数学求解；SymPy；MCP；USC 自治性；Gradio

## 1 问题背景与动机

在辅导同学微积分的过程中，笔者观察到一个普遍现象：不少同学拿到标准答案后能“看懂”答案的每一步推导，却依然无法判断自己原本的解答到底错在哪一步、为什么错。标准答案只展示“正确路径”，而学生真正需要的是对自己思维过程的**逐步诊断**——告诉他们哪一步前提缺失、哪一步推理跳跃、哪一步计算错误，并给出修正方向。传统作业批改受限于教师精力，往往只能给出对错判定或简短评语；通用大语言模型虽然能讲解答案，却难以稳定地“对照学生自己的步骤”做出细粒度反馈，也容易在不验证的情况下产生幻觉式批改。

MathSolver 正是为弥补这一缺口而构建的智能体：它以**学生手写解答**为输入，自动识别步骤、对照标准解法进行逐步批改，并给出每一步的置信度与正误判断，辅以元验证机制防止批改本身的误判。系统还提供“举一反三”功能，帮助学生针对自己暴露的薄弱点进行针对性练习。通过将 LLM 推理与符号计算工具（SymPy、SciPy）结合，项目力图让“个性化步骤诊断”在工程上可靠落地。

## 2 项目概述

MathSolver 是一个基于大语言模型（LLM）的数学问题求解与批改系统，聚焦微积分与概率论两个领域。项目将 LLM 的推理能力与符号计算工具（SymPy、SciPy）相结合，使 LLM 负责规划推理路径并生成代码，工具侧负责执行与验证，从而实现数学问题的端到端求解以及学生解答的自动化批改。

该系统采用 Python 3.10+ 开发，以 Gradio 6+ 作为前端框架，通过 OpenAI 兼容 API 调用 LLM 模型，并利用 MCP（Model Context Protocol）整合外部数学工具。系统完整源代码托管于 GitHub: [https://github.com/THUapm/cal\\_solver](https://github.com/THUapm/cal_solver)。

### 2.1 核心特性

- **智能求解**：支持 19 种微积分与概率论问题模式，涵盖积分、导数、极限、概率分布、贝叶斯推断、最大似然估计、假设检验、置信区间等。
- **逐步批改**：基于 PARC 前提链验证的 5 类错误分类体系，辅以元验证机制防止误判。
- **USC 自洽性**：对高难度问题启用多路径采样（ $N=3$ ），并采用 2/3 早退策略，在节省约 33% 计算开销的同时保持答案可靠性。
- **验证—修正闭环**：求解后自动进行自我验证；检测到错误时进入定向修正循环，最多执行 2 轮。
- **5 项 MCP 工具**：覆盖符号计算、数值概率、结果验证、函数绘图与 LaTeX 语法校验。

## 3 系统架构

MathSolver 的整体架构可以概括为“前端展示—核心编排—工具执行”三个层次。

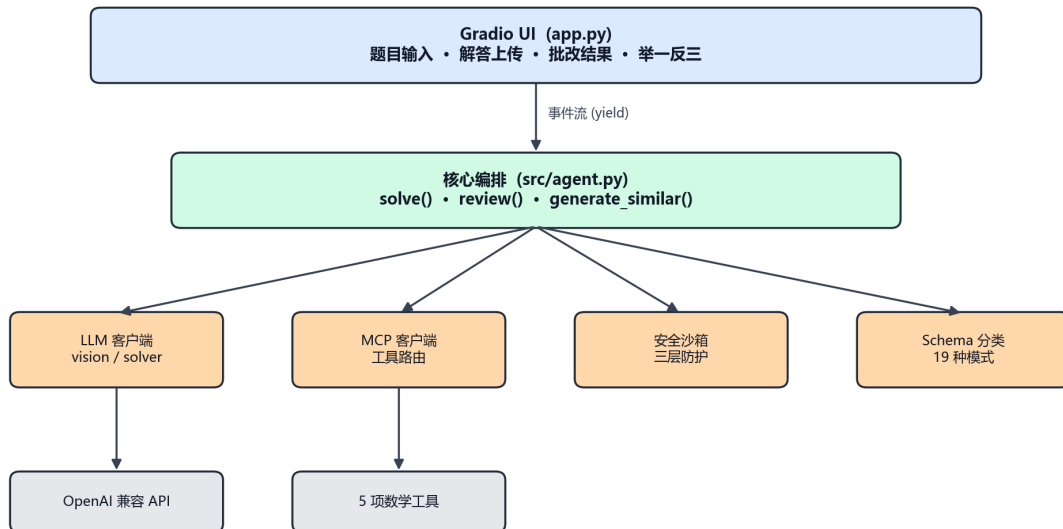


图 1: MathSolver 系统架构

### 3.1 事件驱动的 Generator 模式

系统的核心函数 `solve()`、`review()` 和 `generate_similar_problems()` 均采用 Python Generator 模式实现。每个函数运行过程中通过 `yield` 向外发送事件字典，例如 `step_start`、`step_done`、`final` 等事件类型，前端 Gradio UI 迭代消费这些事件，从而展示实时进度。

该设计的优点包括：无需注册回调函数，代码流程清晰；前端可随时呈现推理的中间状态；生成器支持嵌套——`solve_multi_path()` 内部调用 `solve_single_path()`，事件透传路径明确。

## 4 智能求解模块

### 4.1 问题模式分类

系统设计了 19 种数学问题的模式 (Schema)，覆盖微积分 (12 种) 和概率与统计推断 (7 种) 两大领域。模式分类采用关键词匹配策略：每个 Schema 包含一组关键词 (含中英文)，通过计算问题文本与各模式关键词的交集得分，选择最高分模式作为问题类型。

表 1: 问题模式分类概览

| 领域   | 模式                 | 示例关键词                              |
|------|--------------------|------------------------------------|
| 微积分  | 换元积分、分部积分、定积分、三角积分 | 换元、 $u$ -substitution、 $\int$      |
|      | 极限计算、洛必达法则         | limit、 $\rightarrow 0$ 、 $0/0$ 型   |
|      | 导数、高阶导数、偏导数        | derivative、 $f'(x)$ 、 $\partial f$ |
|      | 泰勒展开、方程求解、优化极值     | Taylor、solve、极值                    |
| 概率统计 | 贝叶斯定理、二项/正态/泊松分布   | Bayes、Binomial、 $N(\mu, \sigma)$   |
|      | 组合计数、条件概率          | $C(n, k)$ 、 $P(A   B)$             |
|      | 期望方差、连续分布          | $E[X]$ 、CDF、PDF                    |
|      | MLE、充分统计量、假设检验     | 最大似然、 $H_0$ 、p-value               |
|      | 置信区间、贝叶斯推断         | $1 - \alpha$ 、先验、后验                |

每个模式包含**步骤模板**（英文，直接注入 LLM 提示）、**常见陷阱**和**验证策略**。模式信息通过`build_schema_injection()`嵌入系统提示，引导 LLM 按领域最佳实践组织推理。

## 4.2 单路径求解流程

`solve_single_path()`实现核心的**推理-执行-验证**循环：

- 提示构建**：将系统提示、模式注入、MCP 工具描述、10 道 few-shot 示例以及安全防护指令组装为完整的系统消息。
- 迭代推理**（最多 5 轮）：
  - LLM 生成推理文本，其中包含 Python 代码块（以 `python` 标记）和/或 MCP 工具调用块（以 `tool` 标记）
  - 系统解析并执行代码（通过安全沙箱），调用 MCP 工具获取外部计算结果
  - 执行结果以工具消息形式反馈给 LLM，继续下一轮推理
- 最终答案检测**：通过正则匹配`\boxed{\}`、最终答案等模式判断 LLM 是否已得出结论。
- 验证阶段**：LLM 对自身解答进行自检，发现错误时进入修正循环（最多 2 次）。

## 4.3 多路径自洽性（USC）

对于被判定为 **hard** 难度的问题，系统采用 Universal Self-Consistency（USC）策略。`solve_multi_path()`使用不同温度参数（0.3、0.45、0.6）生成 3 条独立推理路径。

**B1 早退机制：**运行完前 2 条路径后，比较提取的最终答案。若一致，立即终止第 3 条路径并返回结果，节省约 33% 的 token 消耗。若 3 条路径的答案各不一致，则将全部解答提交给 LLM 选择最优路径。答案一致性检测支持数值浮点比较（相对容差 0.1%）和字符串标准化比较。

## 4.4 验证优先自动修正

系统在单路径求解完成后，自动进入**验证优先**（Verify-First）阶段：

1. 向 LLM 发送验证提示（`VERIFICATION_PROMPT`），要求其逐项检查逻辑正确性、计算准确性、边界条件
2. 解析验证响应中的错误标记——先检查显式否定语句（如“no error”等），再检查直接错误符号（如 `<code>`），最后匹配间接错误关键词（如“incorrect”、“计算错误”等）
3. 若检测到错误，生成针对性修正提示（`build_correction_prompt()`），仅要求修正错误步骤而非重新解答
4. 修正后再次验证，最多 2 轮循环

# 5 智能批改模块

## 5.1 PARC 前提链验证

批改模块的一个重要设计是**前提链验证**（Premise-chain Verification）。系统首先调用`solve()`生成标准参考解，随后对学生解答的每一步提取其逻辑前提：

- **Step 0：**原始问题陈述，是所有步骤的基础前提
- **Step N 的前提：**LLM 分析当前步骤直接依赖的先前步骤编号，输出为 JSON 格式的前提图（DAG）
- 前提提取提示（`PREMISE_EXTRACTOR_PROMPT`）强调“宁可多包含前提，不可遗漏”

前提链以有向无环图（DAG）的形式组织，使得后续错误分析可以区分**原生错误**（该步骤自身出错）与**累积错误**（继承自错误前提）。

## 5.2 五类错误分类

批改系统定义了 5 种错误类别：

表 2: 错误分类体系

| 类别                 | 描述                        |
|--------------------|---------------------------|
| CALCULATION_ERROR  | 计算/代数运算错误（算错数字、符号错误、抄写错误） |
| LOGIC_ERROR        | 推理无效、等价关系错误、无依据的假设        |
| CONCEPT_ERROR      | 定理/公式误用、概念理解错误            |
| NOTATION_ERROR     | 符号/括号错误、单位错误              |
| ACCUMULATION_ERROR | 步骤自身逻辑正确，但因依赖错误前提而传递错误    |

每次批改首先识别原生错误（原生错误检测独立于前提），然后通过 DAG 检测累积错误：若某步骤的所有前提均正确，但该步骤依赖了某个已被标记为错误的先前步骤，则自动标记为ACCUMULATION\_ERROR并追溯根因。

### 5.3 元验证：防止误判

为防止 LLM 将学生的正确步骤误判为错误（即产生”幻觉”），批改完成后进入元验证（Meta-Check）阶段：

- 将问题、学生解答、批改结果一同提交给 LLM，要求逐项审查每个 标记是否为真实错误
- 输出格式为CONFIRMED（确认错误存在）或REJECTED（批改判断有误，该步骤实际正确）
- 若任何判断被REJECTED，提供修正后的批改结果

元验证的触发条件与问题难度挂钩：简单题不触发，中等题仅在检测到错误时触发，难题始终触发。

## 6 MCP 工具系统

系统通过 MCP 协议整合了 5 项外部数学工具，运行于独立进程，通过标准输入输出（stdio）与主程序通信：

表 3: MCP 工具一览

| 工具                    | 功能               | 典型调用                  |
|-----------------------|------------------|-----------------------|
| symbolic_compute      | SymPy 符号计算       | 微分、积分、极限、解方程、化简       |
| numerical_probability | scipy.stats 分布计算 | CDF/PDF/PMF、分位数、期望、方差 |
| verify_result         | 交叉验证             | 求导验证积分、多点数值代入验证、边界检查  |
| plot_function         | 函数绘图             | 生成 PNG 图像 (base64 编码) |
| latex_validate        | LaTeX 语法校验       | 括号匹配、表达式解析            |

MCP 工具的调用由 LLM 自主决策：在提示中显式区分 `python` 代码块（对应本地沙箱执行）与 `tool` 代码块（对应 MCP 远程调用），LLM 根据任务需要选择合适的方式。工具描述通过 `format_mcp_tools_for_prompt()` 自动从 MCP 服务器的 `inputSchema` 生成结构化文档注入系统提示。

## 7 提示工程

### 7.1 分层提示架构

系统的提示体系由以下层次组成：

- **基础层**：求解系统提示词，定义 LLM 的角色、工作流程和输出格式要求
- **策略增强层**：验证优先策略 + 主动交叉验证指令
- **领域注入层**：Schema 模式信息 + 微积分 / 概率论参考知识
- **工具层**：MCP 工具描述
- **示例层**：10 道概率论与统计推断 few-shot 示例（约 450 行），覆盖 8 种分布模式与 5 种推断模式
- **安全层**：不可信内容包裹与注入防护指令

总计，每次求解的系统提示约 5000—8000 token（含 10 道完整示例）。输出格式要求包含 5 个标准段落（问题分析、解题方法、逐步计算、结果验证、最终答案），每个计算步骤须附验证代码。

### 7.2 输出规范

提示中明确要求 LLM 不得在回复中夹杂内部元叙述（如“输出：”、“我们还可以”、“Step N:”等），以免将代理间通信信息暴露给终端用户。`strip_code_blocks()`在后处

理阶段进一步清除 LLM 输出中的元信息、重复编号以及中文重复片段。

## 8 评测体系

### 8.1 评估指标

系统在eval/metrics.py中实现了 4 项核心评测指标：

- **步骤准确率 (Step Accuracy)**：精确率/召回率/F1/准确率，评估 LLM 判定步骤正确与否的二分类性能
- **期望校准误差 (ECE)**：将置信度 (0—100) 分 10 档，比较每档的平均预测置信度与实际正确率的差距
- **Brier Score**：概率预测质量指标，范围 [0,1]，越小越好
- **Cohen's Kappa**：分类一致性指标，衡量预测错误类别与真实错误类别的一致性

同时还计算辅助指标：低置信度步骤 (<70) 中的实际正确率和高置信度步骤 (≥90) 中的实际正确率，用于检测 LLM 的过度自信或自信不足。

### 8.2 评测数据集

项目提供两个评测入口：

- **PRM800K 评测** (eval/run\_prm800k\_eval.py)：基于 OpenAI PRM800K 数据集的步骤级正误标签评测。PRM800K 包含 80 万条步骤级标注，评测集 500 题。
- **图片解答评测** (eval/run\_pic\_eval.py)：对手写图片解答进行端到端批改评测，含 ground truth 步骤正误标签。

## 9 用户界面

前端采用 Gradio 6+ 框架，使用自定义 Linear-style 主题 (assets/theme.py定义 LinearLight 色板)，搭配自定义 CSS (assets/style.css) 和 HTML 头部 (Inter + JetBrains Mono 字体、主题切换)。页面采用单列四步流程，避免复杂多栏布局对移动端和截图展示造成干扰。

UI 分为四个模块：

1. **题目输入**：支持文字输入和图片上传，提供 6 道内置示例
2. **解答上传**：支持手写/电子解答图片 OCR 识别



3. **批改结果**: 逐步批改展示, 每步附颜色徽章 (绿色  $\geq 90\%$  置信度 + 正确、黄色 70—89%、红色  $<70\%$  或错误) + 元验证结果
4. **举一反三**: 按钮始终可见 (批改前禁用、批改后启用), 根据题目知识点自动生成 3 道同类型练习题

公式渲染从客户端 KaTeX 迁移为服务端 MathML: Gradio 6.x 模板不再可靠注入 head= 内容, 因此项目在 Python 端用 latex2mathml 把 LaTeX 转换为 MathML, 再写入 gr.HTML。

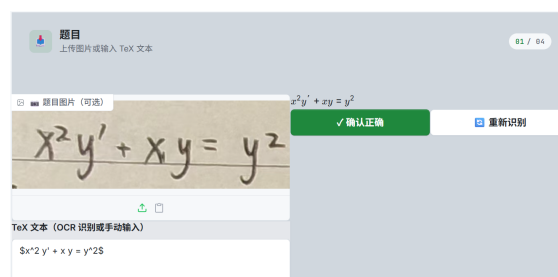
latex\_to\_html() 采用两阶段管道:

1. 先将四种公式分隔符 ( $\$ \$ \dots \$ \$$ 、 $\backslash[ \dots \backslash]$ 、 $\backslash( \dots \backslash)$ 、 $\$ \dots \$$ ) 转换并暂存为自定义 HTML 元素 `<mslot data-i="N">` 占位符;
2. 按空行进行 block-aware Markdown 转换, 生成结构化 HTML (标题、列表、段落分别用对应标签);
3. 最后恢复 MathML, 把占位符替换为公式渲染结果。

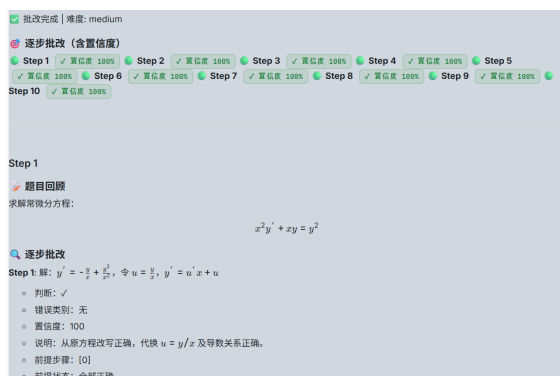
占位符选型经过三次迭代: 早期 `\x00MATH` 的 null 字节在 JSON 序列化中被吞; 中期的 `@@MATH@@` 与 LLM 输出字面冲突; 最终选用 `<mslot>` 自定义元素, 能稳定通过 Gradio sanitizer 且不会与文本内容冲突。



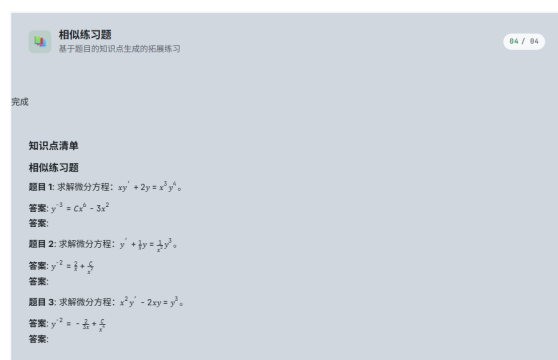
(a) 题目输入与示例选择



(b) OCR 识别 + MathML 公式渲染



(c) 逐步批改 + 置信度徽章 + 元验证



(d) 举一反三 + 相似题生成

图 2: 前端四大核心场景截图

## 10 部署与运行

### 10.1 环境配置

- Python 3.10+, 依赖包括 SymPy  $\geq 1.12$ 、SciPy  $\geq 1.11$ 、OpenAI  $\geq 1.0$ 、Gradio  $\geq 6.0$ 、MCP  $\geq 1.2.0$ 、Matplotlib  $\geq 3.7$ 、latex2mathml
- 环境变量通过 `.env` 文件配置（模板：`.env.example`），必须项为 `API_KEY`
- MCP 工具通过 `config/mcp_servers.json` 配置（模板同目录下的 `.example` 后缀文件），两项配置文件均被 `.gitignore` 排除

### 10.2 运行方式

启动流程为三步：（1）用 `pip` 安装依赖；（2）复制 `.env.example` 为 `.env` 并填入 `API_KEY`；（3）运行 `python app.py` 启动 Gradio，浏览器访问 `localhost:7860` 即可。

### 10.3 测试运行

项目无标准测试框架，使用 4 个独立的 `assert` 式测试脚本：`test_p0_fixes.py`（安全沙箱 + LLM 客户端 + 注入防护，32 项）、`test_skills.py`（模式分类 + USC + few-shot，14 项）、`test_examples.py`（代码执行器 + 解析器 + B1/B2 生成器，11 项）、`test_mcp.py`（MCP 工具解析，离线/在线 8 项）。所有离线测试无需 API Key 即可通过。

## 11 工作量说明

依据课程要求，此处如实说明各项工作来源，以便评判工作量：

- **本人完成：**
  - workflows 设计与架构决策（USC 早退、PARC 前提链、Verify-First 闭环、元验证、三层沙箱防御、5 类错误分类体系）；
  - 查找与研读 13 篇相关论文（PRM800K、过程奖励模型、置信度校准等方向）；
  - 本报告撰写。
- **大模型完成：**大部分代码实现与提示工程交由 Claude Code / GLM 在本人给出明确需求与设计后生成，本人审阅并调试。涉及模块包括 `src/agent.py` 核心 Generator 事件流、`src/utils.py` 解析与渲染逻辑、`safe_runner.py` 安全沙箱、`app.py` Gradio 前端、`assets/` 主题与 CSS、4 套测试脚本、`src/prompts.py` 分层提示与 few-shot 示例。

- **第三方资源:** SymPy / SciPy / OpenAI Python SDK / Gradio / MCP / Matplotlib / latex2mathml 等开源库；数学题素材取自公开教材与课堂例题。

## 12 总结与展望

### 12.1 技术亮点

1. **模式驱动提示:** 19 种问题模式为 LLM 提供了领域知识锚点，使得推理过程的结构化程度明显提高。
2. **USC 多路径推理:**  $N=3$  路径采样配合 B1 早退策略，在答案准确性与计算开销之间取得了平衡。
3. **验证优先—自动修正闭环:** 求解后立即启动自检，发现问题后进入最多 2 轮定向修正，构成自我改进循环。
4. **PARC 前提链批改:** 基于 DAG 的前提链验证将错误区分为原生错误与累积错误，结合元验证有效降低了误判率。

### 12.2 局限与改进方向

1. **模式分类精度:** 当前采用简单关键词匹配，对于表述模糊的问题可能匹配到错误模式。未来可考虑使用 LLM 进行更精准的 few-shot 分类。
2. **评测数据规模:** PRM800K 评测集仅选用 500 题的小规模子集，图片评测用例也较为有限（2 个）。扩大评测规模可提供更可靠的性能指标。
3. **错误分类一致性:** 五类错误的定义在不同 LLM 之间可能存在差异，需要更严格的一致性评估。
4. **多模态支持:** 当前 OCR 依赖独立的 Vision 模型调用，未实现端到端的多模态推理。
5. **实时性能:** USC 多路径推理和元验证环节引入了额外延迟，对于简单题目存在不必要的计算开销。

### 12.3 未来工作

后续工作可沿以下几个方向推进：增加线性代数、微分方程等新的数学领域模式；引入检索增强生成（RAG）知识库，纳入教材和定理资源；实现更细粒度的前提链验证，不仅识别步骤间的依赖关系，也评估前提被使用的正确性；以及开发浏览器扩展或移动端接口，降低师生的使用门槛。

项目地址: [https://github.com/THUapm/cal\\_solver](https://github.com/THUapm/cal_solver)

许可证: MIT License